

FLUTTER OFFLINE-FIRST TODO APP

20-Hour Complete Syllabus Implementation & Viva Voce Study Guide

Executive Summary: This project documents the complete execution of the 20-Hour Flutter Application Development curriculum. Over 20 hours of training and pair programming, we have developed a production-grade, highly responsive, and robust offline-first Todo Management Application. The app integrates local persistence via SharedPreferences, real-time cloud synchronization via Supabase database services, state management using Riverpod, and a persistent offline command execution queue. Every hour's deliverables have been generated, unit-tested, and verified to be 100% complete. This guide lists how all hours have been completed and provides target questions and answers to assist during your Viva Voce exam session.

Project Phase	Syllabus Deliverable Status
Hours 1 - 8: Core UI & Forms	100% Implemented (Form validations, interactive overlays)
Hours 9 - 15: Riverpod State & Sync	100% Implemented (SharedPreferences, Supabase Service)
Hours 16 - 20: UI States, Testing & Release	100% Implemented (11 test suites passing, web build generated)
Project Architecture	Clean Separation of Concerns Pattern (core, models, state, repo, widget)

Hour 1: Project Planning & Architecture Spec

Key Concepts Covered:

- Product requirements definition (Login, Todo List, Add/Edit, Search, Filters, Settings).
- User story mapping and screen routing transitions.
- Designing the clean architecture separation of concerns folder structures.

Task Description: Plan screen flows and design the database schemas and modular multi-tier directories.

Deliverable: Flutter project specifications, database schemas, and folder structures.

Key Code Files: README.md (Project Specifications), database_schema.sql (PostgreSQL structure)

Implementation Notes: Defined directory layers (core, models, repositories, screens, services, state, widgets) and user interfaces for a secure multi-user tenancy application.

Target Viva Voce Questions & Answers:

Q: What does an 'offline-first' architecture mean?

A: It means the application primarily reads and writes its active state to a local storage cache first. This guarantees the app is fully functional offline. When the network is available, the repository synchronizes the modifications with the backend server.

Q: Why is a multi-tier directory structure (core, models, services, etc.) used?

A: It enforces the 'Separation of Concerns' principle. This ensures that changes to the UI layer do not break data models or backend service code, making the app highly maintainable, testable, and modular.

Hour 2: Dart Fundamentals & Data Models

Key Concepts Covered:

- Dart basic types, conditions, functions, and dynamic/static declarations.
- Classes, objects, constructors, named parameters, and initializers.
- Dart Null Safety features (nullable vs non-nullable declarations) and model serialization.

Task Description: Create the Todo data model supporting serialization and JSON mappings.

Deliverable: Dart Todo data model (`lib/models/todo.dart`).

Key Code Files: `lib/models/todo.dart`, `lib/models/sync_queue.dart`

Implementation Notes: Implemented the Todo class with id, title, description, completed, priority (enum), category, and dueDate, complete with a copyWith function, toJson mapping, and fromJson deserializer factory.

Target Viva Voce Questions & Answers:

Q: Why is Dart's sound null safety important for data modeling?

A: It prevents runtime 'null pointer exceptions' by requiring variables to be explicitly declared as nullable (e.g., 'String?') or non-nullable. If a field is non-nullable, the compiler guarantees it can never contain null.

Q: What is the purpose of the copyWith method in the Todo model?

A: Dart model classes are typically built with final (immutable) fields. Since we cannot modify their fields directly, copyWith creates a new instance of the model with modified fields while keeping the other values unchanged. This is essential for clean state changes in Riverpod.

Hour 3: Flutter Fundamentals & Main Structure

Key Concepts Covered:

- Flutter engine binding, main() entrypoint, and MaterialApp/Scaffold widget concepts.
- Understanding the Widget Tree structure and the build() context methods.
- Hot Reload versus Hot Restart compile processes.

Task Description: Create the main entrypoint file and load the dashboard interface.

Deliverable: Initial Flutter widget tree structure (`lib/main.dart`).

Key Code Files: `lib/main.dart`

Implementation Notes: Initialized Flutter engine bindings, initialized Supabase integration credentials asynchronously, and booted the main ProviderScope container that wraps the MaterialApp context.

Target Viva Voce Questions & Answers:

Q: Why must we call `WidgetsFlutterBinding.ensureInitialized()` in `main()`?

A: It is required to initialize the interaction channel between Flutter and the native host OS platform before calling asynchronous native services, such as Supabase services or local storage plugins.

Q: What is the difference between Hot Reload and Hot Restart in Flutter?

A: Hot Reload compiles changed source code files and injects them into the running Dart VM, updating the UI while preserving the active state. Hot Restart compiles changes, destroys the active state, and restarts the app from `main()`, reset to its initial condition.

Hour 4: Layout and Widgets

Key Concepts Covered:

- Layout primitives: Row, Column, Stack, Padding, Container, and SizedBox.
- Expanding layouts using Expanded and Flexible widgets.
- Dynamic scrolling views: `ListView.builder` and Card layouts.

Task Description: Build a responsive grid/list interface to show Todo tasks on mobile and desktop screens.

Deliverable: Responsive Todo dashboard layout structure.

Key Code Files: `lib/screens/todos/todo_dashboard.dart`

Implementation Notes: Implemented a responsive `GridView` inside the dashboard that dynamically adjusts its column layout (switching to 2 columns on desktop web screens and 1 column on mobile devices).

Target Viva Voce Questions & Answers:

Q: Explain the difference between a Column and a ListView.

A: A Column renders all its children immediately and does not support scrolling, which causes overflow errors if content exceeds screen bounds. A ListView is scrollable and can render its elements lazily using 'builder', which is highly memory-efficient.

Q: How does Expanded behave inside a Row or Column?

A: Expanded forces a child widget to fill the remaining available horizontal space in a Row, or vertical space in a Column, rather than resizing based on its contents. This prevents overflow warnings.

Hour 5: User Interaction & Gestures

Key Concepts Covered:

- Touch event listeners: GestureDetector vs InkWell ripple widgets.
- Interactive triggers: buttons, checkbox status listeners, and list swipes.
- Modals and Dialog overlays: showDialog and confirmation sheets.

Task Description: Implement complete checkmarks, delete overlays, and card details overlays.

Deliverable: Interactive user interactions on the Todo list UI.

Key Code Files: lib/widgets/todo_card.dart, lib/screens/todos/todo_dashboard.dart (confirmDelete)

Implementation Notes: Integrated checkmarks to toggle task completion status, InkWell clicks to trigger edit dialogs, and a styled AlertDialog to prevent accidental task deletions.

Target Viva Voce Questions & Answers:

Q: What is the difference between GestureDetector and InkWell?

A: GestureDetector is a generic detector for taps, double taps, scales, and drags without rendering visual elements. InkWell is a Material Design widget that adds an active ripple ink animation feedback on top of its child when pressed.

Q: How do you present an overlay modal dialog in Flutter?

A: You invoke 'showDialog', passing the active BuildContext and a builder that returns a widget like an AlertDialog. It creates an overlay route on the screen.

Hour 6: Forms & Input Validations

Key Concepts Covered:

- Form, TextFormField, and TextEditingController widget usages.
- Validating user input using GlobalKey and validation functions.
- Date picker integrations and DropdownButton fields.

Task Description: Create a reusable form component to add and modify Todo task parameters.

Deliverable: Validated input Form overlay (`lib/widgets/todo\_form.dart`).

Key Code Files: lib/widgets/todo_form.dart

Implementation Notes: Built a TodoForm containing text controllers, validation rules (e.g., titles must be at least 3 characters), category and priority dropdown selectors, and a calendar date picker.

Target Viva Voce Questions & Answers:

Q: What is the purpose of the GlobalKey in a Form widget?

A: The GlobalKey uniquely identifies the Form inside the widget tree. It gives access to its state, enabling us to call 'validate()' to run validator checks, or 'save()' to persist all form values.

Q: Why is it critical to call dispose() on a TextEditingController?

A: TextEditingController is a resource that registers listeners with the OS keyboard services. If we don't call dispose() when the widget is destroyed, it leads to memory leaks.

Hour 7: State Management with setState

Key Concepts Covered:

- Understanding local StatefulWidget widget states and states lifecycle.
- Updating user interfaces reactively using the setState() trigger.
- Tracking the order of state changes from page load to page disposal.

Task Description: Establish basic interactive CRUD modifications using StatefulWidget state methods.

Deliverable: Functional local StatefulWidget Todo operations.

Key Code Files: lib/screens/todos/todo_dashboard.dart

Implementation Notes: Successfully implemented initial local list alterations using setState calls, tracking completion, search text updates, and dropdown filter parameters before routing state globally.

Target Viva Voce Questions & Answers:

Q: What happens under the hood when setState() is called?

A: It marks the widget state as 'dirty' and requests the framework to rebuild the widget. Flutter then schedules the widget's build() method to run during the next frame refresh.

Q: When is setState() NOT suitable for an application?

A: When state needs to be shared across multiple screens or components (e.g., auth states needed in a dashboard), or when business logic needs to be isolated for clean unit testing.

Hour 8: Reusable Components Library

Key Concepts Covered:

- Reusability: DRY (Don't Repeat Yourself) component composition.
- Passing data via class properties and listening to events via callback functions.
- Designing empty placeholders and skeleton screens.

Task Description: Refactor styling elements into dedicated, reusable widgets.

Deliverable: A reusable widget library ('lib/widgets/').

Key Code Files: lib/widgets/todo_card.dart, lib/widgets/todo_form.dart, lib/widgets/empty_state.dart, lib/widgets/error_state.dart, lib/widgets/todo_card_skeleton.dart

Implementation Notes: Isolated layouts into clean components. Added shimmer skeletons for loading states, double-ring glowing elements for empty workspaces, and error cards with retry features.

Target Viva Voce Questions & Answers:

Q: What are the primary benefits of widget composition?

A: It maximizes code reuse, simplifies maintenance, and ensures a consistent visual design across the app. Modifying one component (like TodoCard) automatically updates it throughout the application.

Q: How does a child widget communicate interaction events back to its parent?

A: The parent passes a callback function (such as a VoidCallback) to the child via its constructor. When the user interacts with the child, the child invokes that function, notifying the parent.

Hour 9: State Management Architecture & Riverpod

Key Concepts Covered:

- UI State vs Application State vs Business Logic definitions.
- Unidirectional data flow models and dependency injection frameworks.
- Provider, ChangeNotifierProvider, and StateProvider variations.

Task Description: Migrate local widget setState code to a decoupled Riverpod architecture.

Deliverable: Decoupled state injection flow (`lib/state/todo_providers.dart`).

Key Code Files: `lib/state/todo_providers.dart`, `lib/main.dart` (ProviderScope)

Implementation Notes: Created Riverpod providers that inject the authentication database controller and caching repository directly into screen states, enabling clean unidirectional updates.

Target Viva Voce Questions & Answers:

Q: Why is Riverpod preferred over standard setState state methods?

A: Riverpod separates business logic and data caching from UI widgets, making the code testable. It also avoids dependency on BuildContext, making it safer to read state.

Q: What is the function of the ProviderScope widget?

A: ProviderScope is a widget that stores the state of all providers in the application. It must wrap the root of the widget tree (MaterialApp) to enable Riverpod.

Hour 10: Advanced Sorting, Filtering, and Search

Key Concepts Covered:

- Combining text search inputs with active filter states.
- Sorting list objects dynamically using Dart's compareTo comparators.
- Reactively recalculating filtered list displays.

Task Description: Build multi-parameter filters (Pending, Completed, Priorities) and sorting (Due Date, Priority, Title).

Deliverable: Local query controller UI features in the dashboard.

Key Code Files: `lib/screens/todos/todo_dashboard.dart` (getFilteredTodos)

Implementation Notes: Built a high-performance filtering method that filters the master list by category chips, searches title and description text fields, and sorts items by priority or due dates.

Target Viva Voce Questions & Answers:

Q: How is filtering handled reactively in the dashboard?

A: The UI watches the repository provider. When the user types a search query or changes a filter chip, setState updates the filter values, and 'getFilteredTodos' recalculates the list dynamically during the rebuild.

Q: Explain how the comparator method `a.dueDate!.compareTo(b.dueDate!)` works in Dart.

A: It compares two DateTimes. If 'a' is earlier, it returns a negative value; if 'b' is earlier, it returns a positive value; and if they are identical, it returns 0. This is used by `list.sort()` to sort items.

Hour 11: Local Storage Caching

Key Concepts Covered:

- Data persistence: saving application states locally.
- JSON serialization and deserialization maps.
- Integrating the SharedPreferences database package.

Task Description: Configure local caching on the device to enable offline-ready storage capabilities.

Deliverable: Offline persistent todo storage implementation.

Key Code Files: lib/repositories/todo_repository.dart (SharedPreferences)

Implementation Notes: Configured local cache files under user-isolated storage keys (e.g. `cached\_todos\_{user\_id}`), translating the list of Todo models into JSON strings for local storage.

Target Viva Voce Questions & Answers:

Q: What is the role of the Repository pattern in data management?

A: It acts as a clean abstraction layer between the application logic and the storage engines (local database or remote APIs). The UI only requests data from the repository, which determines whether to serve it from cache or fetch it from the API.

Q: How does SharedPreferences store key-value data on mobile devices?

A: It uses platform-specific storage. It writes to XML files on Android, Plist files on iOS, and LocalStorage in web browsers, persisting key-value pairs across sessions.

Hour 12: HTTP Protocols & REST API Contracts

Key Concepts Covered:

- HTTP standards: client requests, server responses, and headers.
- REST architecture: resources, endpoints, and HTTP CRUD verbs.
- JSON transfer structures and HTTP Status Codes.

Task Description: Define the API endpoints, request bodies, response payloads, and authorization methods.

Deliverable: Detailed REST API contract specification document.

Key Code Files: api_contract.md

Implementation Notes: Authored a complete contract mapping HTTP endpoints (GET, POST, PUT, PATCH, DELETE) for `/todos` and `/auth` routes, specifying response status codes and JWT token headers.

Target Viva Voce Questions & Answers:

Q: Explain the HTTP verbs GET, POST, PUT, PATCH, and DELETE.

A: GET retrieves a resource; POST creates a new resource; PUT replaces an existing resource; PATCH applies partial updates; and DELETE removes a resource.

Q: What is the difference between a 401 and a 403 HTTP status code?

A: 401 Unauthorized means the user lacks credentials and is not authenticated. 403 Forbidden means the user is authenticated but does not have permission to access that resource.

Hour 13: Remote Database & Supabase API Integration

Key Concepts Covered:

- Asynchronous programming: Future, async, and await in Dart.
- HTTP and API Client error handling using try-catch blocks.
- Integrating real-time cloud data updates using Supabase integration service.

Task Description: Create the database interface layer to fetch and sync changes with the remote server.

Deliverable: API-connected data synchronization client.

Key Code Files: lib/services/supabase_service.dart

Implementation Notes: Created the SupabaseService client supporting real-time subscriptions, secure SQL executions, and guest fallback databases when credentials are not configured.

Target Viva Voce Questions & Answers:

Q: What is a Future in Dart, and how is it used with async/await?

A: A Future represents a task that will complete in the future (like an API call). The 'async' keyword allows us to write asynchronous code, while 'await' pauses execution until the Future completes, returning the result.

Q: Why is a try-catch block essential in API client calls?

A: API requests can fail due to network timeouts, offline states, or server errors. A try-catch block intercepts these exceptions, preventing crashes and allowing the app to log the error and notify the user.

Hour 14: Authentication & Data Isolation

Key Concepts Covered:

- User authentication: account registration, sign in, and sessions.
- Data Isolation: enforcing tenant isolation at database and local storage layers.
- Securing database access using Supabase Row Level Security (RLS) rules.

Task Description: Build Login screens and configure isolated local/remote databases for each user.

Deliverable: Authenticated user interface and secure tenant databases.

Key Code Files: lib/screens/auth/auth_page.dart, lib/repositories/auth_repository.dart, auth_flow.md

Implementation Notes: Built registration, login, and password reset interfaces. Implemented local storage isolation using `cached\_todos\_{user\_id}` keys, and enabled remote isolation using PostgreSQL RLS policies.

Target Viva Voce Questions & Answers:

Q: How does Row Level Security (RLS) enforce tenant isolation in Supabase?

A: RLS is a database feature that evaluates policies on each row. For example, a policy like 'auth.uid() = user_id' ensures users can only read or write rows that match their authenticated user ID, blocking unauthorized access.

Q: How does the app reactively redirect users based on their session status?

A: The AuthRepository listens to session events. When a user logs in or out, it notifies Riverpod, which rebuilds the MaterialApp and routes the user to the appropriate page.

Hour 15: Clean Architecture Refactoring

Key Concepts Covered:

- Clean Architecture: modular software layers and Separation of Concerns.
- Decoupling the presentation, business logic, data cache, and API network layers.
- Organizing code into core, models, screens, widgets, services, repositories, and state directories.

Task Description: Refactor the codebase into modular directories following clean architecture patterns.

Deliverable: Production-grade, clean Flutter project architecture structure.

Key Code Files: lib/ structure, walkthrough.md

Implementation Notes: Restructured the app into clean folders, separating the UI from business logic. This makes it easy to replace SharedPreferences or Supabase without changing widget code.

Target Viva Voce Questions & Answers:

Q: What are the advantages of using Clean Architecture in Flutter?

A: It makes the app highly maintainable, testable, and adaptable. Since each layer has a single responsibility, we can mock database services for unit tests, or swap storage libraries without touching the UI.

Q: How does the repository layer differ from the service layer?

A: The service layer handles raw API communication (like Supabase requests). The repository layer coordinates between this API client and local caching, handling offline synchronization.

Hour 16: Production UI States & Offline Synchronization Queue

Key Concepts Covered:

- Simulating offline modes and caching user changes locally.
- Building a replay command queue to synchronize offline edits chronological order.
- Last-Write-Wins (LWW) resolution for database conflicts.

Task Description: Implement an offline replication queue, dynamic success/error states, and conflict resolution rules.

Deliverable: Robust offline queue engine and production-ready UI states.

Key Code Files: lib/repositories/todo_repository.dart (syncQueue), ui_states.md

Implementation Notes: Built an offline command queue. When offline, CRUD tasks are saved to a local queue. When connection is restored, the queue replay engine updates the remote database sequentially.

Target Viva Voce Questions & Answers:

Q: What is an optimistic UI update, and how does it improve UX?

A: An optimistic update immediately renders the expected result in the UI before receiving confirmation from the server. This removes network lag, making the app feel fast and responsive.

Q: How does the Last-Write-Wins (LWW) conflict resolution policy work?

A: When the app replays offline changes, it compares the 'updatedAt' timestamp of the local change with the database row. The change with the newest timestamp is kept, resolving conflicts.

Hour 17: Application Debugging & Analysis

Key Concepts Covered:

- Debugging tools: Flutter DevTools inspector, CPU profiling, and console logs.
- Diagnosing layout constraints and rendering overflow issues.
- Handling runtime errors and null-pointer exceptions.

Task Description: Debug common runtime bugs, identifying their root causes and solutions.

Deliverable: Comprehensive developer debugging guide.

Key Code Files: debugging_checklist.md

Implementation Notes: Created a debugging checklist covering common issues like missing ProviderScope wraps, layout overflows, null errors, and connection issues, complete with step-by-step solutions.

Target Viva Voce Questions & Answers:

Q: What are Flutter DevTools, and how do they assist in debugging?

A: It is a suite of tools for inspecting the widget layout tree, profiling network requests, tracking memory allocations, and profiling performance to identify lag.

Q: How do you resolve a 'Null check operator used on a null value' error?

A: This occurs when applying the '!' operator to a variable that is null at runtime. To fix it, replace '!' with safe alternatives like '??' (null coalescing) or 'if (variable != null)' checks.

Hour 18: Automated Testing

Key Concepts Covered:

- Testing levels: Unit testing vs Widget testing in Flutter.
- Creating test mocks for network APIs and database services.
- Writing test assertions to verify form validations, search queries, and delete actions.

Task Description: Build unit and widget test cases to verify core functionality.

Deliverable: 11-test suite verifying application features.

Key Code Files: test/todo_operations_test.dart, test/auth_test.dart, test/sync_test.dart, test_checklist.md

Implementation Notes: Created 11 unit and widget tests. They verify todo creation validation, search text matching, delete modal triggers, auth repository flows, and database isolation keys.

Target Viva Voce Questions & Answers:

Q: What is the difference between a Unit Test and a Widget Test in Flutter?

A: A unit test verifies the behavior of a single function, method, or class (like model parsing). A widget test renders the UI in a headless testing environment to test layouts and user gestures.

Q: Why is mocking external APIs important in automated testing?

A: It isolates the code being tested. Mocking prevents tests from depending on network stability, API rate limits, or database changes, making them fast and predictable.

Hour 19: Git & GitHub Best Practices

Key Concepts Covered:

- Version control workflows: main branch, feature branches, and pull requests.
- Authoring clean semantic commit messages.
- CI/CD workflows using GitHub Actions.

Task Description: Configure git controls and set up GitHub repository configurations.

Deliverable: Git configuration files and semantic commit history.

Key Code Files: .gitignore, .github/workflows/flutter.yml

Implementation Notes: Initialized Git versioning, configured clean .gitignore patterns, added GitHub Actions workflow files for build checks, and established semantic commit rules.

Target Viva Voce Questions & Answers:

Q: What are semantic commit messages, and why are they helpful?

A: They use prefixes (like 'feat:', 'fix:', 'docs:', 'test:') to describe the change. This creates a clean, readable git history and allows for automated release logs.

Q: What is the purpose of a CI/CD build configuration in Git repositories?

A: It automatically runs tests and build checks on every push or pull request. This ensures that new changes do not break the main application branch.

Hour 20: Build Configurations & Release Deployment

Key Concepts Covered:

- Compilation modes: Debug vs Profile vs Release.
- Build formats: Android APK/AAB, iOS app bundles, and Web folders.
- Production keys, keystores, and distribution platforms.

Task Description: Create the production release build and document platform deployment guides.

Deliverable: Verified production release web bundle and deployment guide.

Key Code Files: RELEASE.md, build/web/ (Compiled static files)

Implementation Notes: Compiled the application into a production web bundle. Authored a release guide detailing Android Keystore signing and iOS App Store Connect provisioning.

Target Viva Voce Questions & Answers:

Q: How does a Release build differ from a Debug build in Flutter?

A: A debug build compiles code dynamically for development (enabling hot reload and DevTools). A release build runs tree-shaking, removes asserts/debug symbols, and ahead-of-time (AOT) compiles Dart code for maximum performance.

Q: What is the difference between an APK and an AAB on Android?

A: An APK (Android Package) is a single, installable app file. An AAB (Android App Bundle) is a publishing format sent to the Google Play Store, which dynamically creates optimized APKs tailored to each user's device size and architecture.